WEEK 4

The MapReduce Paradigm & Apache Spark Core

Batch Processing at Scale

The Functional Model
Map, shuffle & reduce over a cluster

Spark Core
RDDs, DataFrames & lazy evaluation

Catalyst & Tungsten
From query plan to raw speed

CDLE • MEIC/MEIM • ISEL

One Laptop, or Four Hundred Machines?

Count how often each product appears in ten years of server logs: 50 TB, sitting on 400 disks (125GB of data per disk)

ONE MACHINE

≈ 3 days

for a single pass at 200 MB/s
one disk failure → start again from zero

→

400 MACHINES, IN PARALLEL

≈ 10 min

the data is already on their disks
400× the disks, 400× the throughput

The catch someone has to split the data, ship the code to it, survive machines dying mid-job, and merge 400 partial answers correctly. In 2003 every team wrote that part themselves.

*This week: a programming model in which you write two small functions (**map** and **reduce**) and the framework does all the above. Same code on one laptop or a thousand nodes.*

Contents

Everything for week 4 — the MapReduce paradigm, Spark Core, and your first AI-assisted Spark pipelines

01 The MapReduce Paradigm

A functional model of distributed computation
Map, **shuffle**, **reduce** — in code
Why it changed everything — and its limits
From **MapReduce** to **Spark**

02 Apache Spark Core

Architecture: driver, executors, cluster manager
RDDs: resilient distributed datasets
Transformations, actions & lazy evaluation
From RDDs to DataFrames & Datasets

03 Under the Hood

The **Catalyst** optimiser: four phases
Tungsten: memory & code generation
Choosing an API: RDD vs DataFrame vs Dataset
Your first Spark pipeline, end to end

04 Lab — Week 4

First Spark pipelines on the sandbox
AI codegen with test-first validation
Your week-4 checklist
Next week: shuffles, joins & performance

Scenario — Ten Years of Logs

A counting problem that is trivial on one file — and hard on ten billion

The Question

"How many times does each product appear in ten years of server logs — 50 TB spread over 400 machines?"

- The logic is trivial: read, split, count per key
- One machine scans **`~200 MB/s`** — a single pass takes ~3 days
- The data already lives on 400 disks — moving it to one node is the real cost
- And machines will fail mid-job — restarting from zero is not an option

The Four Hard Problems

1 **Partition** cut 50 TB into pieces and decide which machine owns each

2 **Ship compute** move the counting code to the data, never the data to the code (scan ≈ 10 min)

3 **Survive failures** with 400 disks, one dies mid-job; the result must still be right

4 **Merge** 400 partial counts become one correct answer per product

The lesson Google's 2004 answer: two functions (**map** and **reduce)** hide the cluster. You write them; the framework does the rest.

NEW TERMS

- **`TB`** terabyte, 10^{12} bytes
- **`Throughput`** bytes read per second (~200 MB/s per disk)
- **`Key`** the field you count by (here, the product id)
- **`Node`** one machine in the cluster

The Functional Model

1 — Map
(key, value) → list of (new key, value)

2 — Shuffle
list of (key, value) → (key, list of values)

3 — Reduce
(key, list of values) → (key, one result)

Word count: three moves, two functions

```
// one input record: (offset, line) : (0, "step by step")

// map – pure f., run once per record; splits run in parallel
map(_, line) → (step,1) (by,1) (step,1)

// shuffle – group by key across the network: the costly step
(step,[1,1]) (by,[1])

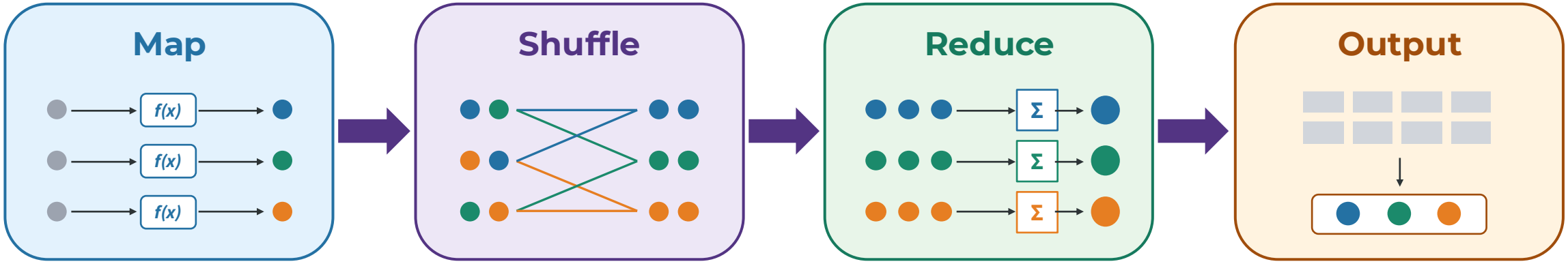
// reduce – one result per key
reduce(step,[1,1]) → (step,2)    reduce(by,[1]) → (by,1)
```

Why two functions are enough

- ✓ You write **map** and **reduce**; the engine shuffles
- ✓ Pure (deterministic and with no side effects) functions re-run safely: retry ≠ corruption
- ✓ One job scales from one node to thousands
- ✓ Counts, sorts, aggregations and joins can all be expressed
- ✓ Engine owns partitioning, scheduling, recovery

Restrict the programming model, and parallelism, distribution and fault tolerance come for free
(after Dean & Ghemawat, MapReduce, OSDI 2004)

The Paradigm in One Picture



1. Stateless & parallel *2. Redistributed by key* *3. Per-key aggregation* *4. One logical result*

This is the paradigm behind Spark

Express computation as functional transformations — let the engine distribute, schedule and recover.

NEW TERMS

`Dot` one record; grey = raw, coloured = keyed

`Colour` the key (blue, green, orange = three distinct keys)

MapReduce in Code

```
# word count — the “hello world” of big data
def map_fn(_, line):
  for w in line.split(): emit(w, 1)

def reduce_fn(word, counts):
  emit(word, sum(counts))

run_job(map_fn, reduce_fn, "/logs/*",
                           "/out/wordcount")
# result: one file per reducer

# yours: two pure functions
# engine's: split, shuffle, retry, write output
```

The Vocabulary

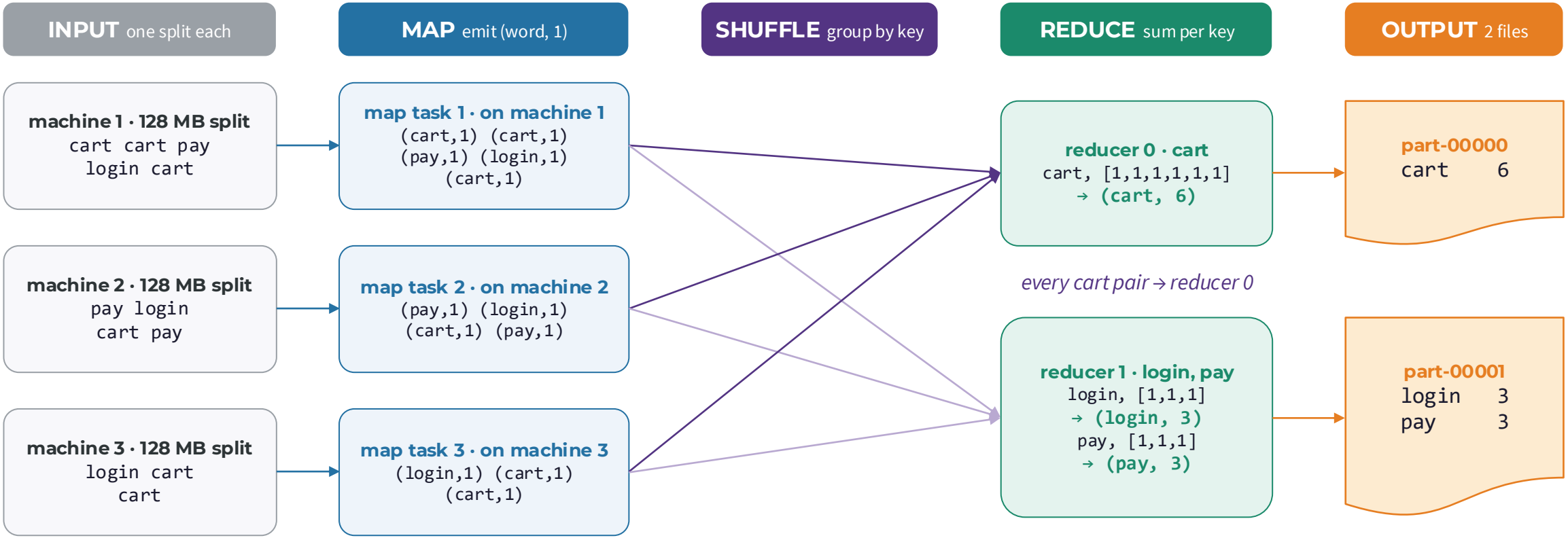
- **Split** → one chunk of input, one map task each
- **Mapper** → emits (key, value) pairs per record
- **Shuffle** → groups pairs by key across the cluster
- **Reducer** → folds each key's values into a result
- **Combiner** → mini-reduce on the map side — saves network
- **Job / task** → one whole run / one unit of work

Hadoop (2006) brought MapReduce to everyone, but every chained job goes through disk. Enter Spark.

NEW TERMS

`emit(k, v)` sends one (key, value) pair to the framework: from map → shuffle, from reduce → output file
Hadoop open-source MapReduce + HDFS (2006), still in use
Spark keeps data in memory between steps (Berkeley, 2009)

Word Count — the Job, Step by Step



M = 3 map tasks, one per split, set by the input size
Combiner: *map task 1 ships (cart,3), not three (cart,1)*

all-to-all over the network
hash(key) mod R → reducer

R = 2 reducers, user-chosen, any free machine
one file per reducer; M and R scale independently

Map and reduce are yours. Splits, the shuffle, retries and writing the output are the framework's — and the shuffle is where the time goes.
One machine holds many splits; it runs one map task per free slot (≈ core).

From MapReduce to Spark

Why the model that started it all was generalised into the engine we use today

1

2004 · The breakthrough

MapReduce hides distribution behind two functions; Hadoop (2006) makes the model open source — web-scale processing for everyone.

2

The pain

Every stage writes to disk; multi-step pipelines become chains of rigid jobs — slow, verbose and hard to iterate on.

3

2010 · Spark's bet

Keep working sets in memory, generalise map/reduce into ~80 composable operators, and recover from failure through lineage.

4

2014+ · Spark Core today

DataFrames, the Catalyst optimiser and Tungsten execution — declarative pipelines compiled into efficient distributed code.

Why this matters

- Spark did not abandon MapReduce — it generalised it. Map, shuffle and reduce still live inside every Spark job you will write.
- This week we read §2–3 of the original MapReduce paper (Dean & Ghemawat, 2004).

NEW TERMS

`Working set` data a job reuses; Spark keeps it in RAM

`Operator` one building block: map, filter, join…

`Lineage` the chain of operators that built a dataset

`Declarative` state the result you want, not the steps

What is Spark Core?

The engine under every Spark library: a distributed collection, a scheduler and a functional API

Why It Won

- In-memory working sets — 10–100× faster than disk-based MapReduce
- ~80 composable operators instead of two — pipelines read like code. Slide 7's word count, plus a filter:

```
lines.flatMap(split).map(w → (w,1))
     .reduceByKey(+).filter((w,n) → n > 100)
```

- One engine for SQL, streaming, ML & graphs
- Lineage-based recovery: recompute only the lost piece, never the whole job

What Core Gives You

- RDDs — the distributed collection everything builds on
- A scheduler that turns your program into parallel tasks
- Lazy evaluation — the whole plan is known before anything runs
- The base for Spark SQL, Structured Streaming and MLlib

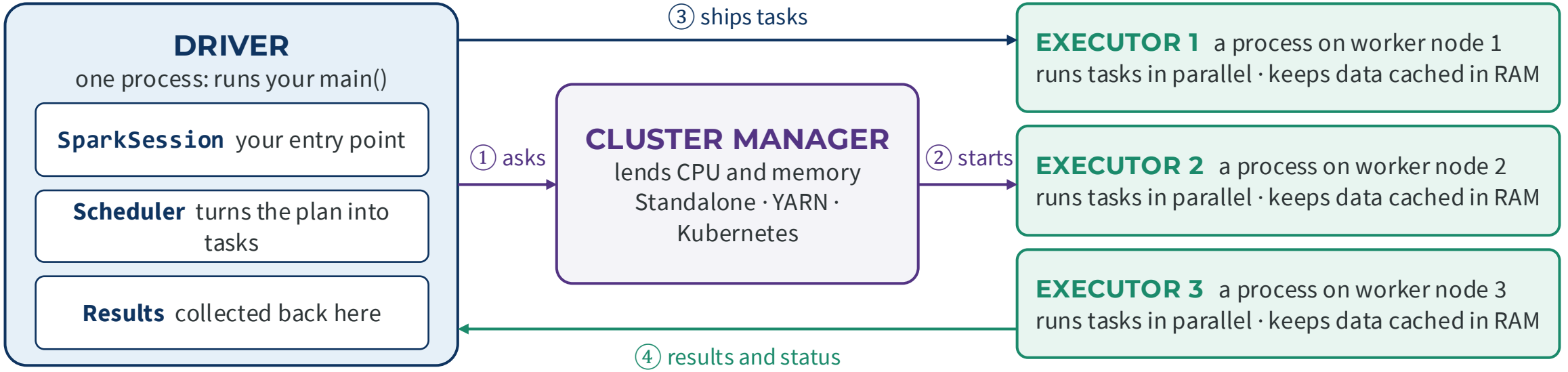
NEW TERMS

DAG (Directed Acyclic Graph) a plan of operators as a graph with no loops
ML Machine Learning
Lazy evaluation plan first, run when a result is asked for
RDD Resilient Distributed Datasets
Spark SQL · MLlib libraries for tables and ML built on Core
Structured Streaming the Core-based library for live data

Coming up in this block

RDDs | DataFrames | Lazy Eval | Catalyst | Tungsten

Spark Runtime — Who Does What



The driver plans and ships tasks, executors run them, the cluster manager only lends machines.
Every Spark UI screen maps onto this picture.

Local mode your laptop plays all three roles in one process. Same program; only the cluster manager changes at 1 000 nodes.

Local *one process on your laptop · labs, tests, small data*
Standalone *Spark's own simple manager · clusters that run only Spark*

YARN *Hadoop's manager (2013) · shared Hadoop clusters, on-premises*
Kubernetes *general container platform (2014) · cloud, Spark alongside other services*

NEW TERMS

- `Driver` the process that runs your program and plans work
- `Executor` a worker process that runs tasks and caches data
- `Cluster manager` hands out machines (YARN, Kubernetes…)
- `SparkSession` the object your code uses to talk to Spark

Meet the RDD

Resilient Distributed Dataset (RDD) — an immutable, partitioned collection with a functional API

all input files
→ one RDD
`sc.textFile(…)`
1 partition / 128 MB

= 4 partitions →



What an RDD Is

- **Partitioned** — cut into chunks, each held by one executor
- **Immutable** — transformations create new RDDs, never edit in place
- **Resilient** — its lineage says how to rebuild any lost chunk
- **Lazy** — nothing runs until an action demands a result

What You Do With It

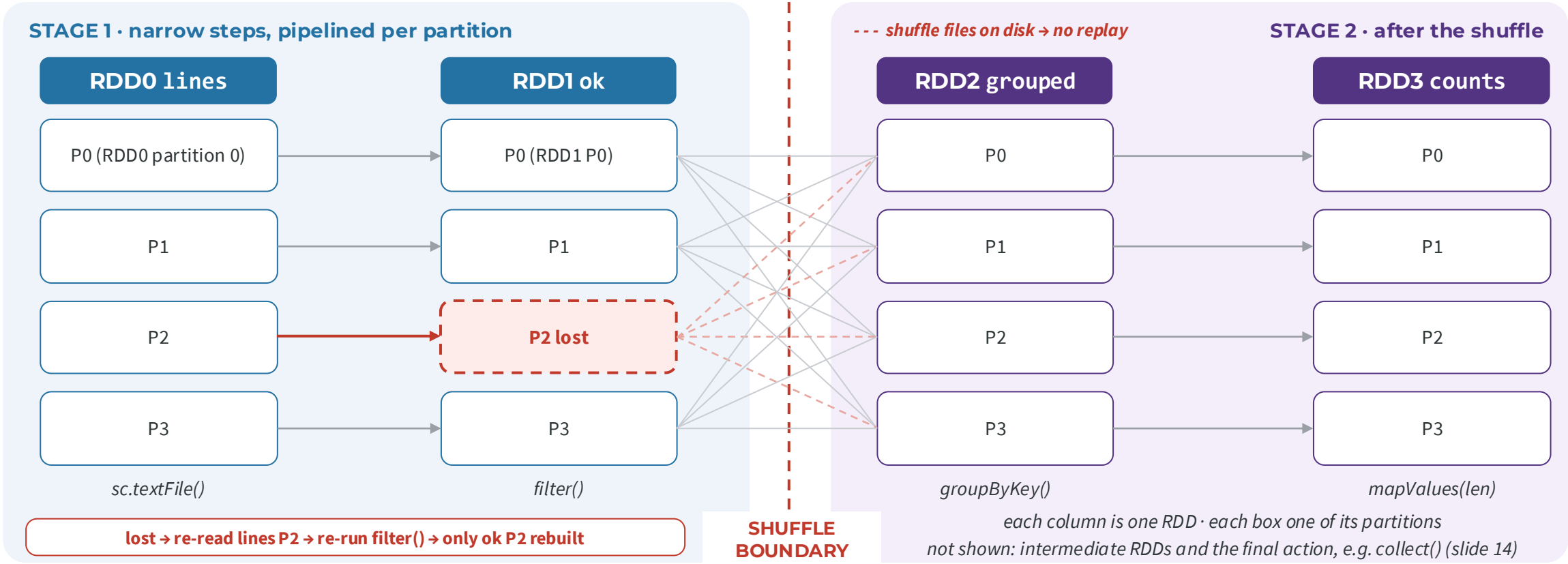
- **Transform**: map, filter, flatMap, reduceByKey, join…
- **Act**: count, collect, take, saveAsTextFile…
- Control: cache() hot data in memory, repartition on purpose
- Recover by recomputing from lineage, not by keeping copies

NEW TERMS

`Partition` one chunk of an RDD, like a MapReduce split; one task each
`Immutable` never changed in place; an operation returns a new RDD
`Lineage` (slide 9) the chain of transformations that built an RDD — replayed to rebuild a lost partition

Partitioned | **Immutable** | **Lazy** | **Resilient** | **In-Memory**

Lineage Is the Recipe



Resilient means Spark stores the recipe, not copies. A lost partition is rebuilt by walking back its lineage to the nearest data that still exists: the source, a cache, shuffle files or a checkpoint. Every shuffle writes Stage 1's output to local disk.

NEW TERMS

- **`Narrow`** one input partition per output; no data moves
- **`Stage`** a run of narrow steps executed together as one pass
- **`Wide`** every input feeds every output; needs a shuffle
- **`Pipelined`** steps applied record by record within one task

Note: ok = the HTTP-200 lines

Transformations vs Actions

Lazy evaluation: transformations only describe; the first action runs the whole plan

Transformations — the plan

```
# nothing executes here
lines = sc.textFile("/logs/*")
ok    = lines.filter(is_200)
pairs = ok.map(url_and_1)
counts = pairs.reduceByKey(add)
# 4 RDDs described, 0 tasks run
```

✓ Fewer passes over the data

Actions — trigger execution

```
# now the whole plan runs
counts.count()
counts.top(20, key=by_count)
counts.saveAsTextFile("/out")
# each action = one Spark job
```

✓ Dead work eliminated

Why laziness wins

```
transform = describe a new RDD
action    = run and return
benefit   = optimise whole plan
beware    = re-run per action
```

✓ Operators fused into stages

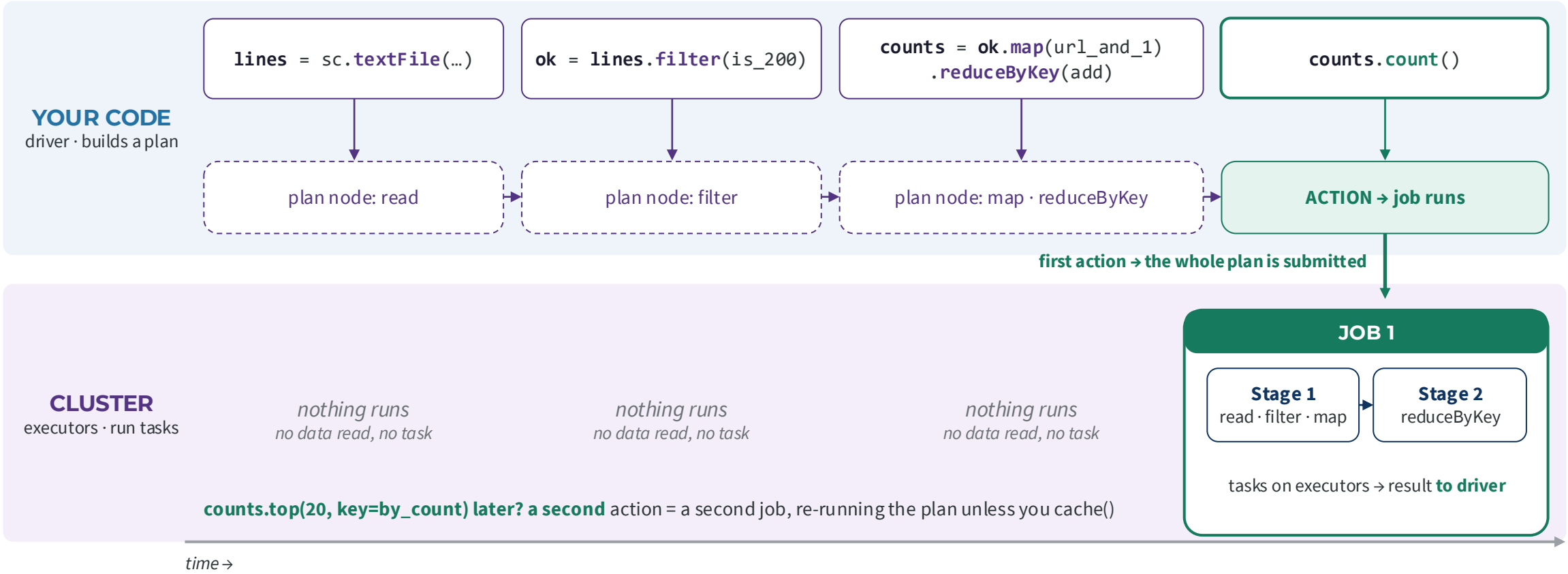
Laziness lets the engine see the whole pipeline before it runs; cache() stops it recomputing a plan you reuse.

NEW TERMS

- **Transformation** describes a new RDD (map, filter…)
- **Action** returns a value or writes (count, save…)
- **Job** the work triggered by one action
- **cache()** keep an RDD in memory after first use
- **sc** the SparkContext: Core's entry point
- **reduceByKey** Spark's name for MapReduce's reduce

```
is_200    = lambda line: line.split()[8] == "200"        # keep successful requests
url_and_1 = lambda line: (line.split()[6], 1)            # one (url, 1) pair per request
by_count  = lambda kv: kv[1]; from operator import add   # sort key for top(20); the reduce function
# 192.0.2.1 - - [24/Sep/2026:10:00:00 +0000] "GET /index.html HTTP/1.1" 200 512  →  [6] /index.html · [8] 200
```

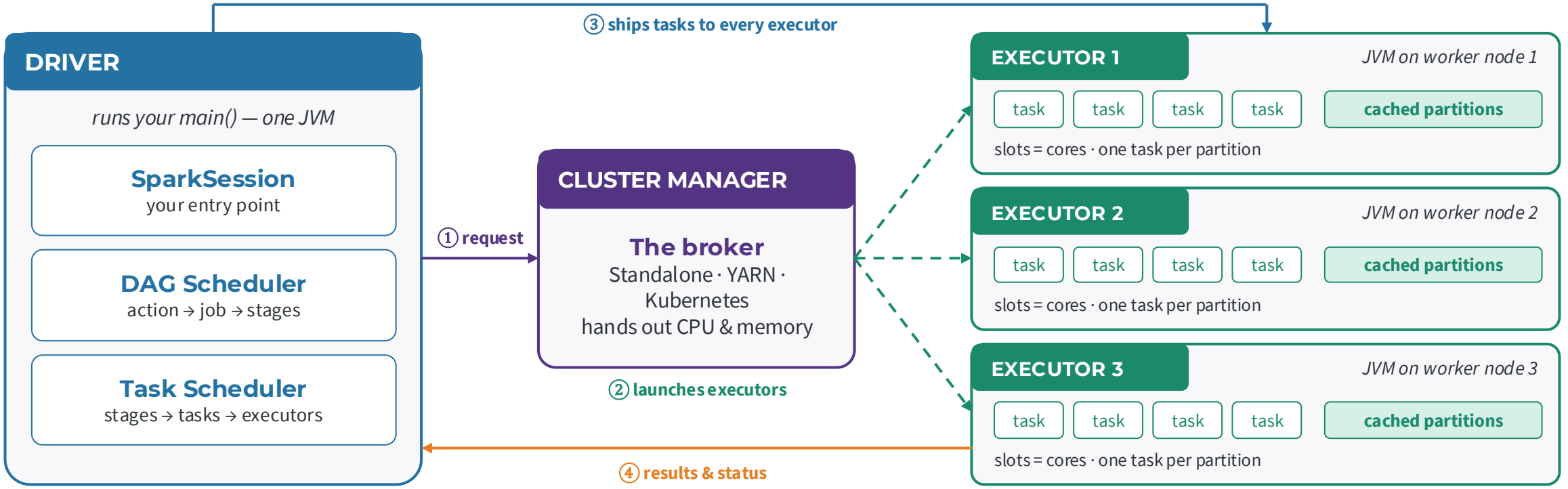
Lazy Until an Action



Transformations only describe. The first action hands the whole plan to the scheduler, so Spark can plan the entire pipeline.

Narrow fuse into one stage: `map` · `filter` · `flatMap` · `mapValues` · `flatMapValues` · `mapPartitions` · `sample` · `union`
Wide shuffle, start a new stage: `reduceByKey` · `groupByKey` · `aggregateByKey` · `combineByKey` · `distinct` · `sortByKey` · `repartition` · `cogroup`

From Action to Tasks



ACTION count(), take(), save…

JOB one per action

STAGES split at every shuffle

TASKS one per partition

One driver plans, N executors run, the manager only lends machines. Every stage ends where data must be shuffled.

NEW TERMS

- **`JVM`** the Java runtime of an executor
- **`Slot`** one executor core; runs one task
- **`Cached partition`** an RDD piece kept in executor RAM – use .cache() when the RDD feeds several actions
- **`Task`** one stage on one partition
- **`DAG`** the plan as a graph of stages, no loops (slide 10)

From opaque - to named columns

RDD functions are opaque to Spark; DataFrames add a schema, so Spark can see, and optimise, what you ask for

RDD — opaque functions

```
# Python functions
ok     = lines.filter(is_200)
pairs  = ok.map(url_and_1)
counts = pairs.reduceByKey(add)
# each step: a black box
counts.take(20)  # runs a job
```

DataFrame — named columns

```
# columns from the schema
counts = (df
    .filter(df.status == 200)
    .groupBy("url")
    .count())  # lazy: no job
counts.take(20)  # runs a job
```

What Spark sees

```
With RDD, Spark sees
  "call a function per line"
With DataFrame, Spark sees
  "status = 200, count per url"
```

✗ **RDD: runs exactly as written**

✓ **DF: unused columns not read**

✓ **DF: filters pushed to source**

*Same result either way. With a schema, **Catalyst** (slide 20) rewrites the plan before any data moves.*

NEW TERMS

- **`DataFrame`** a distributed table with named columns
- **`Schema`** the column names and their types
- **`Opaque function`** code Spark runs but cannot inspect
- **`df`** the log lines parsed into columns

PySpark code: df.status is a shorthand for df["status"]

DataFrame or Dataset

Same table, DF (named) and DS (typed)

```scala
import spark.implicits._
type DataFrame = Dataset[Row]  // Scala
case class LogLine(url: String, status: Int)
val ds = df.as[LogLine]   // LogLine class

// untyped: column name is a string
// a typo fails when the line runs
df.filter($"status" === 200)
// Catalyst reads "status = 200" ✓
// Catalyst can optimise it ✓

// typed: column name is a field of a class
// a typo fails at compile time
ds.filter(_.status == 200)
// this lambda is a black box to Catalyst ✗
```

	RDD opaque objects — your functions	DataFrame named columns — one schema	Dataset typed rows — Scala & Java
OPTIMISER	✗ Blind	✓ Catalyst everywhere	◐ Partial Catalyst, not in lambdas
TYPE-SAFE	✓ Scala/Java only	✗ Runtime only	✓ Compile-time
CONTROL	✓ Full, manual	✗ Delegated	✓ Typed lambdas

In PySpark you will live in DataFrames. RDDs remain underneath; Datasets add compile-time types on the JVM.

NEW TERMS

- **Dataset** a DataFrame with compile-time row types (JVM)
- **Catalyst** Spark's query optimiser (slide 20)

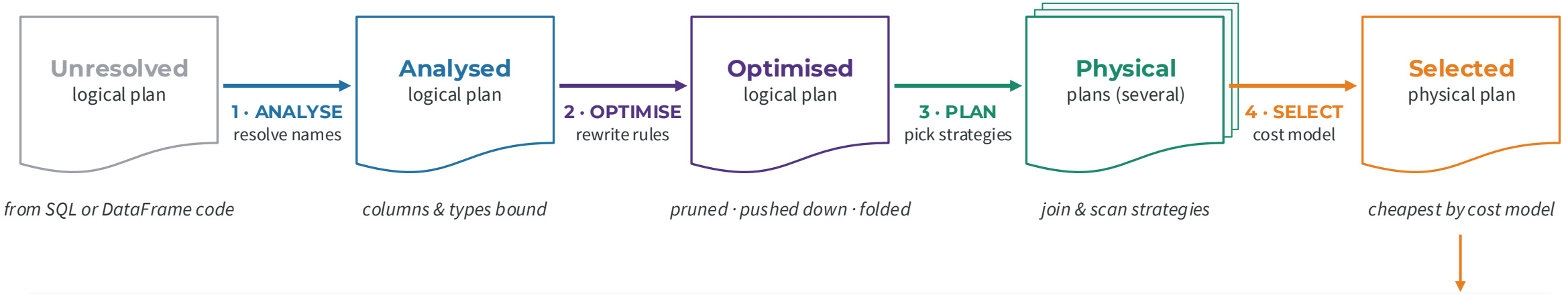
Scala code: $"status is shorthand for col("status")

Choosing Your API

API	What it is	Why you'd use it	Verdict
RDD	`Typed objects · functional ops`	Total control over any data shape	**Low-level**
DataFrame	`Rows with named columns`	Catalyst + Tungsten (slides 20–24) optimise it	**Your default ★**
Dataset	`Typed rows (Scala / Java)`	Compile-time safety + the optimiser	**JVM teams**
Spark SQL	`SQL over the same engine`	Ad-hoc queries; analyst-friendly	**Interop**
pandas API	`pandas syntax on Spark`	Migrating single-node code to a cluster	**Migration**
All of them	`One engine underneath`	Same Catalyst plans, same executors	**Remember**

Rule of thumb: start with DataFrames; drop to RDDs only when you must; use SQL freely, it is the same engine.
`explain()` *prints the plan Spark will run for any of them. Make reading it a habit from today's lab.*
Pandas: the most widely used Python library for working with tables.

Catalyst — From Query to Plan



5 · GENERATE & RUN — handed to Tungsten (slide 22)

Whole-stage codegen
each stage's operators compiled into one tight loop

→

RDD of binary rows
the plan is now ordinary Spark Core code

→

Tasks on executors
one per partition, exactly as on slide 16

`df.explain(True)` prints exactly these plans: Parsed → Analyzed → Optimized → Physical. Next slide shows how to read them.

NEW TERMS

- `Logical plan` what to compute: a tree of operators
- `Physical plan` how to compute it: an algorithm per step
- `Rule` a pattern-based rewrite of the plan tree
- `Cost model` size estimates used to pick the cheapest plan

*You write the what; **Catalyst** rewrites the how. Same code, same result, a plan that moves a fraction of the bytes.*

Reading explain()

The plans **Catalyst** prints for one query, and what to look for in them

counts.explain(True) — four plans for one query

```
# counts = logs.filter(st == 200).groupBy("url").count()
== Parsed Logical Plan ==          names not checked yet
'Aggregate ['url], count ← 'Filter ('st = 200) ← 'UnresolvedRelation
== Analyzed Logical Plan ==        names and types bound
Aggregate [url], count ← Filter (st = 200) ← Relation [url,st,ms]
== Optimized Logical Plan ==
Aggregate [url], count
+- Project [url]                              column pruning
   +- Filter (isnotnull(st) AND st = 200)
      +- Relation parquet
== Physical Plan ==
*(2) HashAggregate(keys=[url], count)
+- Exchange hashpartitioning(url)          the shuffle → new stage
   +- *(1) HashAggregate(partial)       map-side pre-count
      +- *(1) Filter … Project [url]
         +- FileScan parquet PushedFilters:[st = 200]  pushdown
```

How to read it

- **Bottom-up:** the scan is at the bottom, the result at the top
- **Exchange** = a shuffle = a stage boundary; count them
- **PushedFilters** = the filter runs inside the file read
- **HashAggregate(partial)** before the Exchange = the combiner idea from slides 7–8
- ***(n)** = operators fused into one compiled loop, group n (next slide)

NEW TERMS

- **Parquet** columnar file format; reads only needed columns
- **Exchange** the physical operator that performs a shuffle
- **Predicate pushdown** apply a filter at the source, read less
- **Column pruning** read only the columns the query uses

Make explain() a reflex before any heavy run: find the Exchange, check the PushedFilters.

CPU Was the Bottleneck — Tungsten

THE PROBLEM

Rows as Java objects

- every field is a separate object on the heap
- the garbage collector must track them all
- each operator call is a virtual method call per row
- by 2015 CPU, not disk or network, was the limit

TUNGSTEN: THREE TECHNIQUES

1 · Binary rows
one compact byte block per row, off the Java heap; no objects, no GC

2 · Whole-stage codegen
generic operators cost a call per row; one loop written for this schema has none

3 · Vectorised reads
Parquet decoded a column batch at a time; cache-friendly

THE EFFECT

Each CPU cycle counts

- 3.5× less memory per row
- no GC pauses under load
- order-of-magnitude CPU gains
- free: you get it by staying in DataFrames

***Catalyst** decides how few bytes to move; **Tungsten** decides how few CPU cycles each byte costs. Next slide: one row, two ways.*

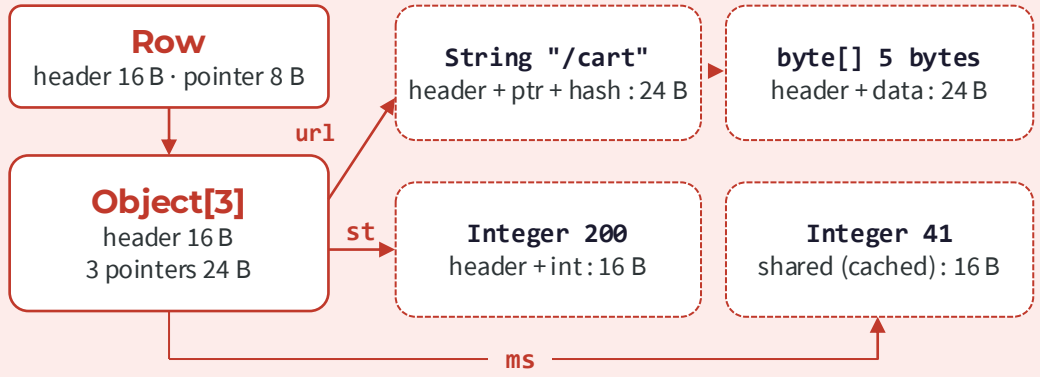
NEW TERMS

`Heap` JVM memory for objects, managed by the GC
`GC` garbage collector: frees objects, pauses the program
`Off-heap` memory Spark manages itself, unseen by the GC
`Codegen` generating and compiling code at run time
`Virtual call` a per-row method call resolved at run time
`Vectorised` one operation on a batch of values, not one a time

Tungsten — One Row, Two Ways

the same record: **`(url = "/cart", st = 200, ms = 41)`**

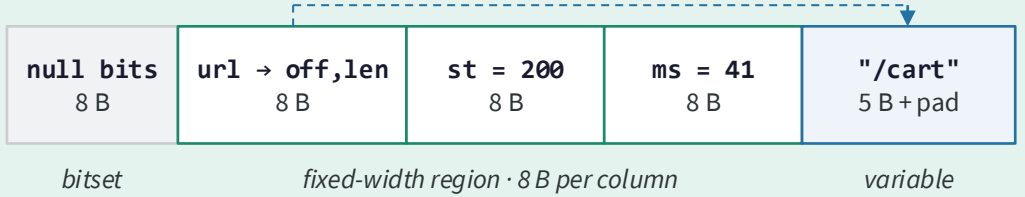
JAVA OBJECTS · what RDDs hold



≈ 145 bytes across 6 heap objects

scattered in memory → cache misses · tracked by the GC → pauses · read by virtual calls → slow

UNSAFEROW · what DataFrames hold



one contiguous block · off-heap or in a byte[] · field i lives at base + 8 + 8·i, no object to follow

UnsafeRow – block (byte[…]) off-heap, outside JVM's garbage collector, use Java's internal Unsafe class.

≈ 40 bytes in one block, 0 objects

contiguous → cache-friendly · invisible to the GC · read by pointer arithmetic in generated code

3.5× less memory
more of the working set fits in RAM

No GC storms
executors stop pausing under load

Codegen-ready
fused loops read fields by offset, no call per row

NEW TERMS

- **`UnsafeRow`** Tungsten's binary row format – java byte[…]
- **`Header`** bookkeeping bytes the JVM adds to every object
- **`Offset`** a field's position in bytes from the row start
- **`Cache miss`** the CPU waits for RAM: data not in its cache

Same record, 145 bytes in six objects or 40 bytes in one block. Generated code reads a field by offset: no object to follow, nothing for the GC.

Tungsten — One Stage, Two Ways

stage *(1) from slide 21: FileScan → Filter st = 200 → Project [url] → partial count

Volcano model
Generic operators — one call per operator per row (simplified)

```
Row next() {                          // Filter
  while (true) {
    Row r = child.next();             // call into Scan
    if (r == null) return null;       // isnotnull
    if ((Boolean) cond.eval(r))       // expression tree
      return r;
  } }
// Project, Aggregate: same pattern, per row
```

Whole-stage codegen
Generated loop — one loop per stage *(1) (simplified)

```
while (scan.hasNext()) {              // FileScan
  UnsafeRow row = scan.next();
  if (row.isNullAt(1)) continue;      // isnotnull
  if (row.getInt(1) != 200) continue; // Filter
  var url = row.getUTF8String(0);     // Project
  map.increment(url);                 // count
}
// one loop: no calls between operators
```

4 operator calls per row → 0

st = 200 → one int comparison

fields read by offset (slide 23)

*counts.explain("codegen") prints the real generated loop for every *(n) stage on slide 21.*

NEW TERMS

- **`Volcano model`** each operator pulls one row at a time
- **`Stage *(n)`** operators fused into one generated loop
- **`JIT`** the JVM compiling hot code into machine code
- **`Janino`** the compiler Spark uses for generated Java

Your First Spark Pipeline

```
# word count again — now in Spark
spark = SparkSession.builder.getOrCreate()
lines = spark.read.text("/logs/*")
words = lines.select(
                explode(split("value", " ")))
counts = words.groupBy("col").count()
counts.write.parquet("/out")

# 4 steps; lazy until write
# Catalyst plans, Tungsten executes
# the same code scales to 50 TB
```

The Anatomy

- **read** → a DataFrame over any storage
- **select / explode** → transformations, still lazy
- **groupBy.count** → the shuffle hides here
- **write** → the action that triggers the job
- **explain()** → always look at the plan before running big
- **Spark UI** → jobs → stages → tasks, live at :4040

This is the shape of every batch pipeline you will write, including the one you build with your assistant in today's lab.

NEW TERMS

- **split(col, " ")** cut a string column into an array of words
- **explode(arr)** one output row per array element (flatMap)
- **select(expr)** pick or compute columns (SQL SELECT)
- **write.parquet** the action: runs the job, saves the result
- **Spark UI** the driver's web page: jobs, stages, tasks

"GET /cart" and "GET /home" become the rows GET, /cart, GET, /home, which become (GET, 2), (/cart, 1), (/home, 1).

Reading List

Ordered by priority — the primary set is the week-4 homework; the secondary set backs one slide each

1 · PRIMARY READING read before lab 5

1. **Dean & Ghemawat — MapReduce: Simplified Data Processing on Large Clusters**
   OSDI 2004 · read §2–3 · backs slides 4–9
2. **Zaharia et al. — Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing**
   NSDI 2012 · §2–4 · backs slides 10–16
3. **Armbrust et al. — Spark SQL: Relational Data Processing in Spark**
   SIGMOD 2015 · §3–4 (Catalyst) · backs slides 17–21
4. **Chambers & Zaharia — Spark: The Definitive Guide**
   O'Reilly 2018 · ch. 2–4 · the lab's PySpark reference

2 · SECONDARY READING dip in per topic

1. **Zaharia et al. — Spark: Cluster Computing with Working Sets**
   HotCloud 2010 · the original in-memory bet (slide 9)
2. **Zaharia et al. — Apache Spark: A Unified Engine for Big Data Processing**
   CACM 2016 · one engine for SQL, streaming, ML (slide 10)
3. **Kleppmann — Designing Data-Intensive Applications, ch. 10**
   O'Reilly 2017 · batch processing in context (slides 2, 4, 7)
4. **Databricks — Project Tungsten & Whole-Stage Code Generation**
   Engineering blog 2015–16 · binary rows, codegen (slides 22–23)
5. **Apache Spark 3.5 docs — RDD & SQL/DataFrame Programming Guides**
   spark.apache.org · API detail for the lab (slides 12–19, 24)
6. **Stonebraker et al. — MapReduce and Parallel DBMSs: Friends or Foes?**
   CACM 2010 · the critique behind “its limits” (slide 9)

Primary titles are in the week-4 checklist (slide 30); all PDFs and links are in the lab repository under readings/week-04.

PART 2 · HANDS-ON

Lab

04